Kth Smallest

Difficulty: **Medium** Accuracy: **35.17%** Submissions: **742K+** Points: **4** Average Time: **25m**

Given an integer array **arr[]** and an integer **k**, your task is to find and return the **kth smallest** element in the given array.

Note: The kth smallest element is determined based on the sorted order of the array.

Examples :

Input: arr[] = [10, 5, 4, 3, 48, 6, 2, 33, 53, 10], k = 4
Output: 5
Explanation: 4th smallest element in the given array is 5.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1121 / 1121

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
import java.util.Arrays;
class Solution {
    public int kthSmallest(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k - 1];
    }
}
```

Custom Input Compile & Run Submit

Minimize the Heights II

Difficulty: **Medium** Accuracy: **15.06%** Submissions: **775K+** Points: **4** Average Time: **25m**

Given an array **arr[]** denoting heights of **n** towers and a positive integer **k**.

For **each** tower, you must perform **exactly one** of the following operations **exactly once**.

- **Increase** the height of the tower by **k**
- **Decrease** the height of the tower by **k**

Find out the **minimum** possible difference between the height of the shortest and tallest towers after you have modified each tower.

You can find a slight modification of the problem here.
Note: It is **compulsory** to increase or decrease the height by k for each tower. After the

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1115 / 1115

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
import java.util.Arrays;

class Solution {
    public int getMinDiff(int[] arr, int k) {
        int n = arr.length;
        if (n == 1) return 0;

        Arrays.sort(arr);

        int ans = arr[n - 1] - arr[0];

        int smallest = arr[0] + k;
        int largest = arr[n - 1] - k;

        for (int i = 1; i < n; i++) {
            int minVal = Math.min(smallest, arr[i] - k);
            int maxVal = Math.max(largest, arr[i - 1] + k);

            if (minVal < 0) continue;

            ans = Math.min(ans, maxVal - minVal);
        }

        return ans;
    }
}
```

Custom Input Compile & Run Submit

Minimum Jumps

Difficulty: **Medium** Accuracy: **11.91%** Submissions: **1.1M** Points: **4**

You are given an array **arr[]** of non-negative numbers. Each number tells you the **maximum number of steps** you can jump forward from that position.

For example:

- If **arr[i] = 3**, you can jump to index **i + 1**, **i + 2**, or **i + 3** from position **i**.
- If **arr[i] = 0**, you **cannot jump forward** from that position.

Your task is to find the **minimum number of jumps** needed to move from the **first** position in the array to the **last** position.

Note: Return **-1** if you can't reach the end of the array.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1120 / 1120

Attempts : Correct / Total

5 / 5

Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int minJumps(int[] arr) {
        int n = arr.length;

        if (arr[0] == 0) return -1;

        int maxReach = arr[0];
        int steps = arr[0];
        int jumps = 1;

        for (int i = 1; i < n; i++) {
            if (i == n - 1) return jumps;

            maxReach = Math.max(maxReach, i + arr[i]);
            steps--;

            if (steps == 0) {
                jumps++;

                if (i >= maxReach) return -1;

                steps = maxReach - i;
            }
        }

        return -1;
    }
}
```

Custom Input Compile & Run Submit

287. Find the Duplicate Number

Solved

Medium | Topics | Companies

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

You must solve the problem **without** modifying the array `nums` and using only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

Example 3:

```
Input: nums = [3,3,3,3,3]
Output: 3
```

Constraints:

- `1 <= n <= 10^5`
- `nums.length == n + 1`
- `1 <= nums[i] <= n`

Code

Java Auto

```java
class Solution {
    public int findDuplicate(int[] nums) {
        int slow = nums[0];
        int fast = nums[0];

        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);

        slow = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
```

Saved Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[1,3,4,2,2]

Output

2

Expected

2

Merge Without Extra Space

Difficulty: **Medium** Accuracy: **32.01%** Submissions: **329K+** Points: **4** Average Time: **20m**

Given two sorted arrays **a[]** and **b[]** of size **n** and **m** respectively, the task is to merge them in sorted order without using any **extra space**. Modify **a[]** so that it contains the first **n** elements and modify **b[]** so that it contains the last **m** elements.

Examples:

Input: a[] = [2, 4, 7, 10], b[] = [2, 3]
Output: a[] = [2, 2, 3, 4], b[] = [7, 10]
Explanation: After merging the two non-decreasing arrays, we get, [2, 2, 3, 4, 7, 10]

Input: a[] = [1, 5, 9, 10, 15, 20], b[] = [2, 3, 8, 13]

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1111 / 1111

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    public void mergeArrays(int a[], int b[]) {
        int n = a.length;
        int m = b.length;

        int gap = n + m;

        while (gap > 0) {
            gap = nextGap(gap);

            int i = 0, j = gap;

            while (j < n + m) {
                if (i < n && j < n) {
                    if (a[i] > a[j]) {
                        int temp = a[i];
                        a[i] = a[j];
                        a[j] = temp;
                    }
                }
                else if (i < n && j >= n) {
                    if (a[i] > b[j - n]) {
                        int temp = a[i];
                        a[i] = b[j - n];
                        b[j - n] = temp;
                    }
                }
                else {
                    if (b[i - n] > b[j - n]) {
                        int temp = b[i - n];
                        b[i - n] = b[j - n];
                        b[j - n] = temp;
                    }
                }

                i++;
                j++;
            }
        }
    }

```

Custom Input Compile & Run Submit

56. Merge Intervals

Solved

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.
```

Example 3:

```
Input: intervals = [[4,7],[1,4]]
Output: [[1,7]]
Explanation: Intervals [1,4] and [4,7] are considered overlapping.
```

Constraints:

- `1 <= intervals.length <= 10^4`
- `intervals[i].length == 2`
- `0 <= start_i <= end_i <= 10^4`

24.5K | 274 | 300 Online

Code

Java | Auto

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length <= 1) {
            return intervals;
        }

        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        List<int[]> merged = new ArrayList<>();

        int[] current = intervals[0];
```

Saved | Ln 8, Col 1

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2 | Case 3

Input

intervals =

```
[[1,3],[2,6],[8,10],[15,18]]
```

Output

```
[[1,6],[8,10],[15,18]]
```

Expected

```
[[1,6],[8,10],[15,18]]
```

Common in 3 Sorted Arrays

Difficulty: **Easy** Accuracy: **22.16%** Submissions: **441K+** Points: **2**

Given three sorted arrays in **non-decreasing** order, print all common elements in **non-decreasing** order across these arrays. If there are no such elements return an empty array. In this case, the output will be -1.

Note: can you handle the duplicates without using any additional Data Structure?

Examples :

Input: arr1 = [1, 5, 10, 20, 40, 80] , arr2 = [6, 7, 20, 80, 100] , arr3 = [3, 4, 15, 20, 30, 70, 80, 120]
Output: [20, 80]

Output Window

Compilation Results Custom Input

Compilation Error

Ask Yogi Bot

```
./GFG.java: 31: error: method commonElements in class Solution cannot be applied to given types;
        List<Integer> res = ob.commonElements(arr, brr, crr);
                            ^
  required: int[],int[],int[],int,int,int
  found:    List<Integer>,List<Integer>,List<Integer>
  reason: actual and formal argument lists differ in length
1 error
```

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public ArrayList<Integer> commonElements(int[] arr1, int[] arr2, int[] arr3) {

        ArrayList<Integer> res = new ArrayList<>();

        int i = 0, j = 0, k = 0;

        while (i < arr1.length && j < arr2.length && k < arr3.length) {

            if (arr1[i] == arr2[j] && arr2[j] == arr3[k]) {

                if (res.size() == 0 || res.get(res.size() - 1) != arr1[i]) {
                    res.add(arr1[i]);
                }

                i++;
                j++;
                k++;
            }
            else if (arr1[i] < arr2[j]) {
                i++;
            }
            else if (arr2[j] < arr3[k]) {
                j++;
            }
            else {
                k++;
            }
        }

        if (res.size() == 0) {
            res.add(-1);
        }

        return res;
    }
}
```

Custom Input Compile & Run Submit

Factorials of large numbers

Difficulty: **Medium** Accuracy: **36.57%** Submissions: **178K+** Points: **4** Average Time: **20m**

Given an integer **n,** find its factorial. Return a list of integers denoting the digits that make up the factorial of n.

Examples:

Input: n = 5
Output: [1, 2, 0]
Explanation: 5! = 1*2*3*4*5 = 120

Input: n = 10
Output: [3, 6, 2, 8, 8, 0, 0]

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1111 / 1111

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
import java.util.ArrayList;

class Solution {
    public static ArrayList<Integer> factorial(int n) {
        ArrayList<Integer> result = new ArrayList<>();

        result.add(1);

        for (int i = 2; i <= n; i++) {
            multiply(result, i);
        }

        return result;
    }

    private static void multiply(ArrayList<Integer> result, int num) {
        int carry = 0;

        for (int i = result.size() - 1; i >= 0; i--) {
            int product = result.get(i) * num + carry;
            result.set(i, product % 10);
            carry = product / 10;
        }

        while (carry > 0) {
            result.add(0, carry % 10);
            carry /= 10;
        }
    }
}
```

Custom Input Compile & Run Submit

Array Subset

Difficulty: **Basic** Accuracy: **44.05%** Submissions: **526K+** Points: **1** Average Time: **20m**

Given two arrays **a[]** and **b[]**, your task is to determine whether **b[]** is a subset of **a[]**.

Examples:

Input: a[] = [11, 7, 1, 13, 21, 3, 7, 3], b[] = [11, 3, 7, 1, 7]
Output: true
Explanation: b[] is a subset of a[]

Input: a[] = [1, 2, 3, 4, 4, 5, 6], b[] = [1, 2, 4]
Output: true
Explanation: b[] is a subset of a[]

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1114 / 1114

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public boolean isSubset(int a[], int b[]) {
        HashMap<Integer, Integer> map = new HashMap<>();

        for (int num : a) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        for (int num : b) {
            if (!map.containsKey(num) || map.get(num) == 0) {
                return false;
            }
            map.put(num, map.get(num) - 1);
        }

        return true;
    }
}
```

Custom Input Compile & Run Submit

Triplet Sum in Array

Difficulty: **Medium** Accuracy: **35.0%** Submissions: **364K+** Points: **4** Average Time: **15m**

Given an array **arr[]** and an integer **target**, determine if there exists a triplet in the array whose sum equals the given **target**.

Return **true** if such a triplet exists, otherwise, return **false**.

Examples:

Input: arr[] = [1, 4, 45, 6, 10, 8], target = 13
Output: true
Explanation: The triplet {1, 4, 8} sums up to 13.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1111 / 1111

Attempts : Correct / Total

2 / 3

Accuracy : 66%

Time Taken

Java (21) Start Timer

```java
import java.util.Arrays;

class Solution {
    public boolean hasTripletSum(int arr[], int target) {
        int n = arr.length;
        Arrays.sort(arr);

        for (int i = 0; i < n - 2; i++) {
            int left = i + 1;
            int right = n - 1;

            while (left < right) {
                int sum = arr[i] + arr[left] + arr[right];

                if (sum == target) {
                    return true;
                }
                else if (sum < target) {
                    left++;
                }
                else {
                    right--;
                }
            }
        }
        return false;
    }
}
```

Custom Input Compile & Run Submit

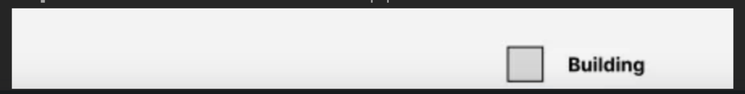
Trapping Rain Water

Difficulty: **Hard** Accuracy: **33.14%** Submissions: **499K+** Points: **8** Average Time: **20m**

Given an array **arr[]** with non-negative integers representing the height of blocks. If the width of each block is 1, compute how much water can be trapped between the blocks during the rainy season.

Examples:

Input: arr[] = [3, 0, 1, 0, 4, 0 2]
Output: 10
Explanation: Total water trapped = 0 + 3 + 2 + 3 + 0 + 2 + 0 = 10 units.



Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1111 / 1111

Attempts : Correct / Total
2 / 2
Accuracy : 100%

Time Taken

Java (21) Start Timer

```java
class Solution {
    public int maxWater(int arr[]) {
        int n = arr.length;
        if (n == 0) return 0;

        int left = 0, right = n - 1;
        int leftMax = 0, rightMax = 0;
        int water = 0;

        while (left <= right) {
            if (arr[left] <= arr[right]) {
                if (arr[left] >= leftMax) {
                    leftMax = arr[left];
                } else {
                    water += leftMax - arr[left];
                }
                left++;
            } else {
                if (arr[right] >= rightMax) {
                    rightMax = arr[right];
                } else {
                    water += rightMax - arr[right];
                }
                right--;
            }
        }

        return water;
    }
}
```

Custom Input Compile & Run Submit